

COE4DK4 Lab 2

Discrete-Event Simulation of Queueing and Network System

400440917-XikaiXu-xu503

400473040-ZihaoZhou- zhouz247

Date: October 4, 2026

Academic Integrity Statement:

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario.

Submitted by [Xikai Xu, 400440917]

Academic Integrity Statement:

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario.

Submitted by [Zihao Zhou, 400473040]

Executive Summary

The laboratory progressed from a single M/D/1 queue to a mixed-traffic quality-of-service model. The measured M/D/1 delay grew nonlinearly near the 2000 packets/s capacity. For the 2000-bit Part 2 packets, 400 packets/s was the largest tested arrival rate satisfying $P(\text{Delay} > 20 \text{ ms}) < 0.02$. The M/D/2 configuration had the same aggregate capacity as M/D/1 but a longer individual service time. In the three-switch network, $p_{12} = 0.5$ balanced the exit links and minimized Source 1 delay. Under FCFS, voice delay exceeded one second near saturation, while non-preemptive voice priority reduced the 22 packets/s voice delay to about 40 ms at the cost of increased data delay.

1. Simulation Methodology

The simulator used an event list containing packet-arrival and transmission-completion events. A packet stored in its arrival time and service time. Total delay was measured at departure as the current simulation time minus the original arrival time. Runtime parameter arrays replaced repeated source edits, so each experiment was compiled once and automatically executed for every operating point and random seed. Results were written to CSV immediately after each run.

2. Code Modifications and Major Implementation Details

Only the model-defining portions of the source are shown below. The complete Simlib support code was retained. These excerpts document what changed and why each modification was required.

2.1 Matching with the requirements

```
#ifndef _SIMPARAMETERS_H_
#define _SIMPARAMETERS_H_

/*****

#define PACKET_ARRIVAL_RATE 1900
#define PACKET_LENGTH 5e2 /* bits */
#define LINK_BIT_RATE 1e6 /* bits per second */
#define RUNLENGTH 10e6 /* packets */

/* Comma separated list of random seeds to run. */
#define RANDOM_SEED_LIST 400440917, 400473040

#define PACKET_XMT_TIME ((double) PACKET_LENGTH/LINK_BIT_RATE)
#define BLIPRATE (RUNLENGTH/1000)

*****/
```

We first modified the experimental parameters in `simplparameters.h`. We configured the simulator with 500-bit packets and a 1 Mbps transmission link for this experiment.

2.2 Part 2 delay-tail counter

Main.h add this parameter to the struct pack

```
45      /* Part 2 statistics */
46      long int exceed_20ms_count;
--
```

Count the number of packets with a delay exceeding 20 ms to calculate $P(\text{delay} > 20 \text{ ms})$.

Packet_transmission.c

```

data->accumulated_delay += packet_delay;
if(packet_delay > 0.020)
{
    data -> exceed_20ms_count++;
}
/* Output activity blip every so often. */
output_progress_msg_to_screen(simulation_run);

```

Delay cannot be recorded at arrival because the packet has not finished service yet. We therefore compute it inside the transmission-completion event, using the difference between the current simulation time and the packet's stored arrival time.

Main.c adding the looping code, to reduce the time needed to compile the code. Noticed post 400, we reduced the scanning resolution, to precisely targeting the 2% edge.

```

53     FILE *csv_file;
54
55     double arrival_rates[] = {
56         300,325,350,375,
57         400,410,420,430,
58         440,450,460,470,
59         480,490
60     };
61
62     int number_of_rates;
63     int rate_index;
64
65     /*
66      * Declare and initialize our random number generator seeds defined in
67      * simparameters.h
68      */
69
70     int j = 0;
71     for (rate_index = 0; rate_index < number_of_rates; rate_index++){
72         j = 0;
73         while ((random_seed = RANDOM_SEEDS[j++]) != 0) {
74
75             simulation_run = simulation_run_new(); /* Create a new simulation run. */
76
77             /*

```

```

-
4  /*****
5
6  #ifndef _SIMPARAMETERS_H_
7  #define _SIMPARAMETERS_H_
8
9  /*****
0
1  // #define PACKET_ARRIVAL_RATE 400 /* packets per second */
2  #define PACKET_LENGTH 2e3 /* bits */
3  #define LINK_BIT_RATE 1e6 /* bits per second */
4  #define RUNLENGTH 10e6 /* packets */
5
6  /* Comma separated list of random seeds to run. */
7  #define RANDOM_SEED_LIST 400440917, 400473040
8
9  #define PACKET_XMT_TIME ((double) PACKET_LENGTH/LINK_BIT_RATE)
0
1  #define DELAY_LIMIT_SECONDS 0.020
2  #define BLIPRATE (RUNLENGTH/1000)
3
4  *****/

```

This the parameters we used for the part 2 experiments.

2.3 Part 3 shared queue and two servers

```

/*****

#include "simlib.h"
#include "simparameters.h"

#define NUMBER_OF_SERVERS 2
*****/

typedef struct _simulation_run_data_
{
    Fifoqueue_Ptr buffer;
    Server_Ptr link[NUMBER_OF_SERVERS];
}

```

Due to the M/D/2 model contains dual 500kbps processing pipeline, we need 2 server/link under main.h

```

110     data.exceed_20ms_delay = 0;
111     data.exceed_20ms_count = 0;
112
113     data.arrival_rate = arrival_rates[rate_index];
114     data.random_seed = random_seed;
115
116     /*
117     * Create the packet buffer and transmission link, decl
118     */
119
120     data.buffer = fifoqueue_new();
121
122     data.link[0] = server_new();
123     data.link[1] = server_new();
124
125     /*
126     * Set the random number generator seed for this run.

```

Part 3 main.c structure, we created 2 parallel servers, to simulate 2 pipe links under servers.

```

1
2     if(server_state(data->link[0]) == FREE)
3     {
4         start_transmission_on_link(
5             simulation_run,
6             new_packet,
7             data->link[0]
8         );
9     }
10    else if(server_state(data->link[1]) == FREE)
11    {
12        start_transmission_on_link(
13            simulation_run,
14            new_packet,
15            data->link[1]
16        );
17    }
18    else
19    {
20        fifoqueue_put(data->buffer, (void *)new_packet);
21    }
22    /*
23    * Create the packet buffer and transmission link, decl

```

In `packet_arrival.c`, we added a two-server check. When a packet arrives, the code tries `link[0]` first, then `link[1]`. If either server is free, the packet starts transmission immediately. Only when both servers are busy does the packet enter the shared FIFO queue. This keeps both links utilized whenever possible, which matches the M/D/2 model.

```

3
4  /*****
5
6  #ifndef _SIMPARAMETERS_H_
7  #define _SIMPARAMETERS_H_
8
9  /*****
0
1  // #define PACKET_ARRIVAL_RATE 400 /* packets per second */
2  #define PACKET_LENGTH 500 /* bits */
3  #define LINK_BIT_RATE 500e3 /* bits per second */
4  #define RUNLENGTH 10e6 /* packets */
5
6  /* Comma separated list of random seeds to run. */
7  #define RANDOM_SEED_LIST 400440917, 400473040
8
9  #define PACKET_XMT_TIME ((double) PACKET_LENGTH/LINK_BIT_RATE)
0
1  #define DELAY_LIMIT_SECONDS 0.020
2  #define BLIPRATE (RUNLENGTH/1000)
3
4  /*****
5
6  #endif /* simparameters.h */

```

500 kbps links shared in one FCFS queue. A packet was queued only when both servers were busy. The transmission-completion event received the actual server pointer as its event attachment, so the next waiting packet could use the server that had just become free.

2.4 Part 4 routing and source-aware statistics

```

Code Blame 39 lines (30 loc) · 706 Bytes
1  #ifndef _MAIN_H_
2  #define _MAIN_H_
3
4  #include "simlib.h"
5  #include "simparameters.h"
6
7  typedef struct _simulation_run_data_
8  {
9      Fifoqueue_Ptr buffer1;
10     Fifoqueue_Ptr buffer2;
11     Fifoqueue_Ptr buffer3;
12
13     Server_Ptr link1;
14     Server_Ptr link2;
15     Server_Ptr link3;
16
17     long int blip_counter;
18     long int arrival_count[3];
19     long int number_of_packets_processed[3];
20     double accumulated_delay[3];
21
22     double p12;
23     unsigned random_seed;
24 } Simulation_Run_Data, *Simulation_Run_Data_Ptr;
25
26 typedef enum {XMTTING, WAITING} Packet_Status;
27
28 typedef struct _packet_
29 {
30     double arrive_time;
31     double service_time;
32     int source_id;
33     int destination_id;
34     Packet_Status status;
35 } Packet, *Packet_Ptr;
36
37 int main(void);
38
39 #endif

```

Under main.h, we create 3 entities of switchers, The three switches each have independent queues and links; end-to-end latency must be aggregated and analyzed based on the original source.

```

5     data.blip_counter = 0;
7     data.p12 = p12_values[p_index];
3     data.random_seed = random_seed;
3     for (i = 0; i < 3; i++) {
3         data.arrival_count[i] = 0;
1         data.number_of_packets_processed[i] = 0;
2         data.accumulated_delay[i] = 0.0;
3     }
1
5     data.buffer1 = fifoqueue_new();
5     data.buffer2 = fifoqueue_new();
7     data.buffer3 = fifoqueue_new();
3     data.link1 = server_new();
3     data.link2 = server_new();
3     data.link3 = server_new();
1
2     random_generator_initialize(random_seed);
3     schedule_packet_arrival_1_event(simulation_run, 0.0);
1     schedule_packet_arrival_2_event(simulation_run, 0.0);
5     schedule_packet_arrival_3_event(simulation_run, 0.0);
5
7     while (total_processed(&data) < RUNLENGTH)
3         simulation_run_execute_event(simulation_run);
3
3     output_results(simulation_run);
1     fprintf(csv_file,
2         "%.1f,%u,%ld,%ld,%ld,%.8f,%.8f,%.8f\n",
3         data.p12,
1         data.random_seed,
5         data.number_of_packets_processed[0],
5         data.number_of_packets_processed[1],
7         data.number_of_packets_processed[2],
3         1000.0 * data.accumulated_delay[0] /
3         data.number_of_packets_processed[0],
3         1000.0 * data.accumulated_delay[1] /
1         data.number_of_packets_processed[1],
2         1000.0 * data.accumulated_delay[2] /
3         data.number_of_packets_processed[2]);
1     fflush(csv_file);
5     cleanup_memory(simulation_run);
5     }
7     }

```

Initialize three switches and schedule three independently arriving flows in main.c; scan p12_values [].

Packets from Source 1 do not finish after Link 1. They are forwarded to either Link 2 or Link 3, so their total delay must be measured after the second hop. We stored source_id in each packet so that delay could still be grouped by the original source. The p12 scan shows how routing probability changes the delay of each source.

```

7     event.event;
1     event.description = description;
2     event.function = function;
3     event.attachment = NULL;
4     return simulation_run_schedule_event(simulation_run, event, event_time);
5 }
6
7 long int schedule_packet_arrival_1_event(Simulation_Run_Ptr simulation_run,
8                                         double event_time)
9 {
10     return schedule_arrival_event(simulation_run, event_time,
11                                   packet_arrival_1_event, "Source 1 Arrival");
12 }
13
14 long int schedule_packet_arrival_2_event(Simulation_Run_Ptr simulation_run,
15                                         double event_time)
16 {
17     return schedule_arrival_event(simulation_run, event_time,
18                                   packet_arrival_2_event, "Source 2 Arrival");
19 }
20
21 long int schedule_packet_arrival_3_event(Simulation_Run_Ptr simulation_run,
22                                         double event_time)
23 {
24     return schedule_arrival_event(simulation_run, event_time,
25                                   packet_arrival_3_event, "Source 3 Arrival");
26 }
27

```

Implement `packet_arrival_1_event`, `_2_event`, and `_3_event` in `packet_arrival.c` and store the `source_id` when the packet is created; the original source must be preserved even after the packet is forwarded to Link 2 or 3 so that latency statistics for Sources 1, 2, and 3 can be correctly grouped.

```

void end_packet_transmission_event(Simulation_Run_Ptr simulation_run, void *ptr)
{
    Simulation_Run_Data_Ptr data;
    Server_Ptr completed_link = (Server_Ptr)ptr;
    Packet_Ptr packet;

    data = (Simulation_Run_Data_Ptr)simulation_run_data(simulation_run);
    packet = (Packet_Ptr)server_get(completed_link);

    if (completed_link == data->link1) {
        /* Keep Link 1 busy with its next queued packet before forwarding. */
        start_next_waiting_packet(simulation_run, data->buffer1, data->link1);

        if (uniform_generator() < data->p12) {
            packet->destination_id = 2;
            send_or_queue(simulation_run, packet, data->link2, data->buffer2);
        } else {
            packet->destination_id = 3;
            send_or_queue(simulation_run, packet, data->link3, data->buffer3);
        }
        return;
    }

    if (completed_link == data->link2) {
        complete_network_packet(simulation_run, data, packet);
        start_next_waiting_packet(simulation_run, data->buffer2, data->link2);
        return;
    }

    if (completed_link == data->link3) {
        complete_network_packet(simulation_run, data, packet);
        start_next_waiting_packet(simulation_run, data->buffer3, data->link3);
        return;
    }
}

```

The completion event in `packet_transmission.c` checks which link finished. If Link 1 finished, the packet is still not complete. It is sent to Link 2 with probability p_{12} , or to Link 3 otherwise. The final delay is only added when Link 2 or Link 3 finishes. For Source 1 packets, this means the measured delay includes both hops, from original arrival to final departure.

2.5 Part 5 mixed traffic under FCFS

```

--
25  ✓ typedef struct _simulation_run_data_
26      {
27          Fifoqueue_Ptr buffer;
28          Server_Ptr link;
29
30          long int blip_counter;
31          long int voice_arrival_count;
32          long int data_arrival_count;
33          long int voice_processed_count;
34          long int data_processed_count;
35
36          double voice_accumulated_delay;
37          double data_accumulated_delay;
38          double data_arrival_rate;
39

```

Voice packets were generated at fixed 20 ms intervals, while data packets followed a Poisson process.

In Part 5, both types joined one common FIFO queue, and the server always took the packet at the front, so the service order was strictly FCFS.

```

47     data.blip_counter = 0;
48     data.voice_arrival_count = 0;
49     data.data_arrival_count = 0;
50     data.voice_processed_count = 0;
51     data.data_processed_count = 0;
52     data.voice_accumulated_delay = 0.0;
53     data.data_accumulated_delay = 0.0;
54     data.data_arrival_rate = data_arrival_rates[rate_index];
55     data.random_seed = random_seed;
56
57     data.buffer = fifoqueue_new();
58     data.link = server_new();
59
60     random_generator_initialize(random_seed);
61     schedule_voice_packet_arrival_event(simulation_run, 0.0);
62     schedule_data_packet_arrival_event(simulation_run, 0.0);
63
64     /* If (data->processed_count) < QUEUE_LENGTH */
60     packet->service_time = VOICE_SERVICE_TIME;
61     packet->type = VOICE_PACKET;

```

packet_delay = simulation_run_get_time(simulation_run) - packet->arrive

```

if (packet->type == VOICE_PACKET) {
    data->voice_processed_count++;
    data->voice_accumulated_delay += packet_delay;
} else {
    data->data_processed_count++;
    data->data_accumulated_delay += packet_delay;
}

xfree(packet);
output_progress_mse_to_screen(simulation_run);

```

In `packet\_transmission.c`, accumulate counts (`completed[type]++`) and delays (`accumulated\_delay[type] += ...`) categorized by `packet->type`; then, calculate the average total delay for voice and data separately within the completion event.

```

3
4     /* Voice: G.711, 64 kbit/s, 20 ms packetization, 62-byte header. */
5     #define VOICE_INTERARRIVAL_TIME 0.020
6     #define VOICE_PACKET_LENGTH 1776.0
7

```

2.6 Part 6 non-preemptive voice priority

```

25  ✓ typedef struct _simulation_run_data_
26      {
27          Fifoqueue_Ptr voice_buffer;
28          Fifoqueue_Ptr data_buffer;
29          Server_Ptr link;
30
31          long int link_counter;

```

In Part 6, we used two separate queues. After each transmission, the code checks the voice queue first and serves the oldest voice packet if one is waiting. Otherwise it serves the oldest data packet. A data packet already in service is never interrupted, so the priority is non-preemptive.

```

52          data.random_seed = random_seed;
53
54          data.voice_buffer = fifoqueue_new();
55          data.data_buffer = fifoqueue_new();
56          data.link = server_new();
57
58          random_generator_initialize(random_seed);

```

Create two independent queues in main.c: `data.voice\_buffer = fifoqueue\_new();` and `data.data\_buffer = fifoqueue\_new();` to initialize the dual-queue structure.

```

void voice_packet_arrival_event( 
    Simulation_Run_Ptr simulation_run,
    void *unused)
{
    Simulation_Run_Data_Ptr data;
    Packet_Ptr packet;
    (void)unused;

    data = (Simulation_Run_Data_Ptr)simulation_run_data(simulation_run);
    data->voice_arrival_count++;

    packet = (Packet_Ptr)xmalloc(sizeof(Packet));
    packet->arrive_time = simulation_run_get_time(simulation_run);
    packet->service_time = VOICE_SERVICE_TIME;
    packet->type = VOICE_PACKET;
    packet->status = WAITING;

    start_or_enqueue(simulation_run, data, packet, data->voice_buffer);

    schedule_voice_packet_arrival_event(
        simulation_run,
        simulation_run_get_time(simulation_run) + VOICE_INTERARRIVAL_TIME);
}

void data_packet_arrival_event( 
    Simulation_Run_Ptr simulation_run,
    void *unused)
{

```

In `packet\_arrival.c`, voice packets enter `voice\_buffer` and data packets enter `data\_buffer`; traffic is diverted based on service type to facilitate priority-based scheduling.

```

if (fifoqueue_size(data->voice_buffer) > 0) {
    next_packet = (Packet_Ptr)fifoqueue_get(data->voice_buffer);
    start_transmission_on_link(simulation_run, next_packet, link);
} else if (fifoqueue_size(data->data_buffer) > 0) {
    next_packet = (Packet_Ptr)fifoqueue_get(data->data_buffer);
    start_transmission_on_link(simulation_run, next_packet, link);
}

```

Implementing non-preemptive priority: Current packet transmission is not interrupted, but voice packets are prioritized for the next transmission.

3. Part 1: M/D/1 Mean Delay

A. Experiment Settings :

Settings	Values
Model	M/D/1, FCFS
Packet Length	5e2 bits
Link Bit Rate	1Mbps
Packet Transmission Times	0.5ms
Rate	10 e 6
Seed	400440917, 400473040
runs	2 * 11

Part 1 used 500-bit packets, Poisson arrivals, one FCFS queue, and a deterministic 1 Mbps link. The 0.5 ms service time corresponds to a 2000 packet/s service capacity. The selected arrival rates cover light, moderate, and near-saturation operation.

B. results

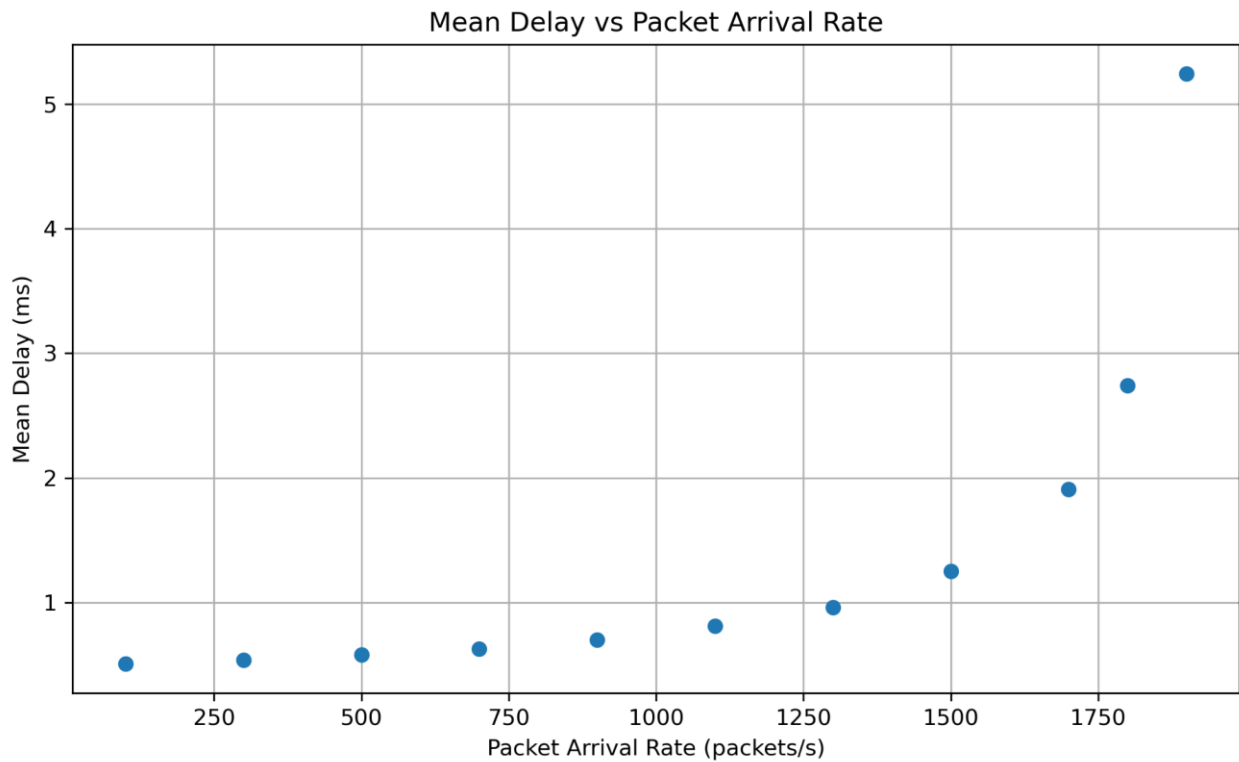


Figure 1. Mean packet delay for the M/D/1 system.

Arrival rate (packets/s)	Mean delay (ms)
100	0.51
300	0.54
500	0.58
700	0.63
900	0.70
1100	0.81
1300	0.96
1500	1.25
1700	1.91
1800	2.74
1900	5.24

The results using seed: 400440917

At 100 packets/s, the mean delay was 0.51 ms, almost equal to the service time. At 1900 packets/s, utilization was 0.95 and delay reached 5.24 ms. The sharp rise near capacity reflects queue accumulation as idle periods become rare.

C. Discussions:

At $\lambda = 300$, the value is approximately 0.54 ms; at 500, ~ 0.58 ms; at 700, ~ 0.63 ms; at 900, ~ 0.70 ms; at 1100, ~ 0.81 ms; at 1300, ~ 0.96 ms; at 1500, ~ 1.25 ms; at 1700, ~ 1.90 – 1.91 ms; at 1800, ~ 2.72 – 2.74 ms; and at 1900, ~ 5.20 – 5.24 ms. This sharp increase as the rate approaches 2,000 packets/second is expected, as the traffic load is nearing $\rho = 1$. At low and moderate arrival rates, repeated measurement results are very similar; differences become slightly more pronounced at the highest loads, where random traffic bursts can cause packets to experience longer waiting times. For instance, at $\lambda = 1,900$ packets/second, the latencies recorded in the CSV were 5.24 ms and 5.20 ms—values that remain very close to each other. This is because the duration of each run is substantial (transmitting 10^6 packets per run), sufficient to smooth out fluctuations caused by the stochastic nature of the arrival process and varying random seeds. Consequently, the average results from multiple runs can be used for each data point in the plot. Overall, the experimental results exhibit the expected characteristics of the M/D/1 model: when the arrival rate is well below the service rate, latency is primarily composed of packet

transmission time; however, as the arrival rate approaches the link capacity (2,000 packets/second), queuing delay becomes the dominant component of total latency, leading to a sharp rise in the average total latency.

4. Part 2: Delay Violation Probability

A. Experiment Settings :

Settings	Values
Model	M/D/1, FCFS
Packet Length	2e3 bits
Link Bit Rate	1Mbps
Packet Transmission Times	2ms
Rate	10 e 6
Seed	400440917, 400473040
runs	2 * 14

Part 2 like Part 1, also uses an M/D/1 queue with FCFS service, but the packet length is increased to 2000 bits. With a 1 Mbps link, the service time becomes 2 ms, and the service capacity is 500 packets/s. The goal is to find the largest tested arrival rate for which the probability of delay exceeding 20 ms stays below 2%.

The arrival rates scanned are 300, 325, 350, 375, 400, 410, 420, 430, 440, 450, 460, 470, 480, and 490 packets/s. The same two seeds, 400440917 and 400473040, were used. Each run started empty and processed 10,000,000 completed packets. No warm-up removal was applied. For every completed packet, the delay was compared with the 20 ms limits, and the number of late packets was recorded. The violation probability was computed as the number of late packets divided by the total completed packets.

The packet length increased to 2000 bits, so service time became 2 ms and server capacity became 500 packets/s. The objective was to find the greatest tested arrival rate satisfying $P(\text{Delay} > 20 \text{ ms}) < 0.02$.

Results:

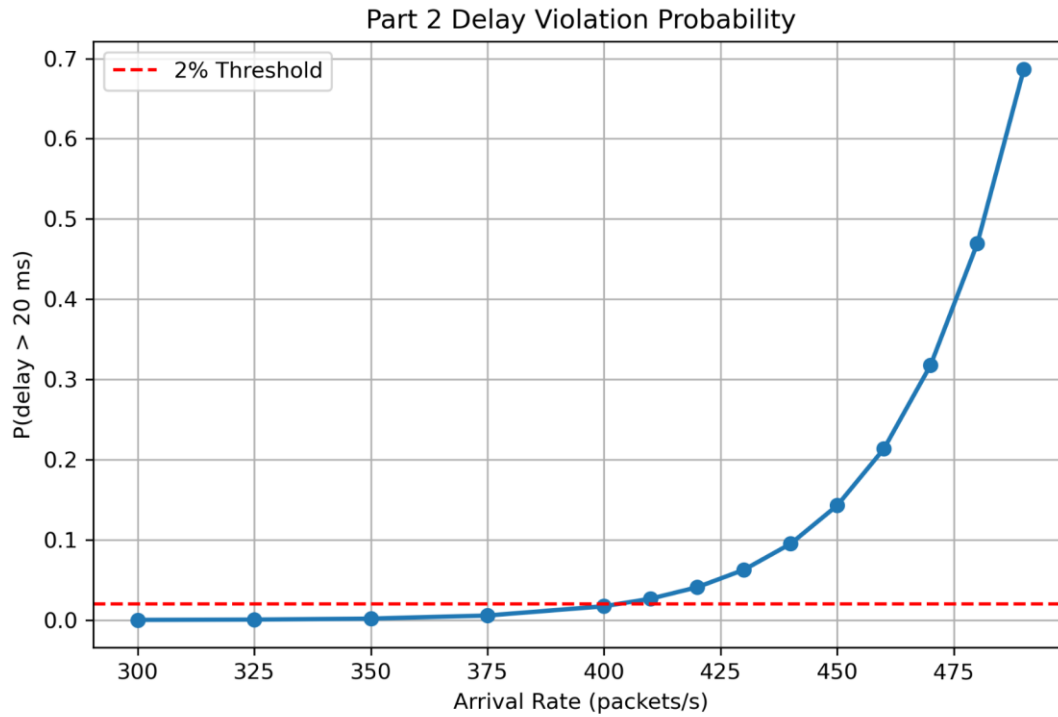


Figure 2. Delay-violation probability with the 2% requirement shown as a dashed line.

The two-seed mean probability was 0.0171607 at 400 packets/s and 0.0265820 at 410 packets/s.

Therefore, 400 packets/s was the largest tested compliant rate. The 2% delay requirement is reached well before the system becomes mathematically unstable.

	lambda	seed	processed	violations	probability	mean_delay_ms
	300	400440917	10000000	1472	0.0001472000	3.498731
	300	400473040	10000000	1450	0.0001450000	3.499141
	325	400440917	10000000	5214	0.0005214000	3.855044
	325	400473040	10000000	5288	0.0005288000	3.855702

350	400440917	10000000	17680	0.0017680000	4.330300
350	400473040	10000000	17714	0.0017714000	4.329479
375	400440917	10000000	56445	0.0056445000	4.993854
375	400473040	10000000	55900	0.0055900000	4.990007
400	400440917	10000000	173807	0.0173807000	5.987735
400	400473040	10000000	169407	0.0169407000	5.976750
410	400440917	10000000	269493	0.0269493000	6.540522
410	400473040	10000000	262147	0.0262147000	6.524226
420	400440917	10000000	414478	0.0414478000	7.232065
420	400473040	10000000	404663	0.0404663000	7.208022
430	400440917	10000000	633104	0.0633104000	8.118812
430	400473040	10000000	620461	0.0620461000	8.085793
440	400440917	10000000	957894	0.0957894000	9.301278
440	400473040	10000000	944087	0.0944087000	9.257015

450	400440917	10000000	1441551	0.1441551000	10.965858
450	400473040	10000000	1417348	0.1417348000	10.888089
460	400440917	10000000	2148291	0.2148291000	13.454395
460	400473040	10000000	2122717	0.2122717000	13.336977
470	400440917	10000000	3193666	0.3193666000	17.614652
470	400473040	10000000	3161669	0.3161669000	17.466780
480	400440917	10000000	4710594	0.4710594000	25.947654
480	400473040	10000000	4672764	0.4672764000	25.802478
490	400440917	10000000	6884063	0.6884063000	50.262003
490	400473040	10000000	6854894	0.6854894000	51.077397

C. Discussion

At $\lambda = 300$ packets/second, the proportions of packets exceeding 20 ms latency for the two random seeds were 0.01472% and 0.01450%, respectively, with average latencies of approximately 3.4987 ms and 3.4991 ms; as the arrival rate increased, both the violation probability and the average latency continued to rise. At $\lambda = 400$ packets/second, the violation probabilities for the two seeds were 1.73807% and 1.69407%, with average latencies of approximately 5.9877 ms and 5.9768 ms; at $\lambda = 410$ packets/second, the violation probabilities rose to 2.69493% and 2.62147%, with average latencies of approximately 6.5405 ms and 6.5242 ms; and at $\lambda = 490$ packets/second, the violation probabilities

reached 68.84063% and 68.54894%, with average latencies of approximately 50.2620 ms and 51.0774 ms.

5. Part 3: M/D/2 Parallel Links

A. Experiment Settings :

Settings	Values
Model	M/D/2, FCFS
Packet Length	5e2 bits
Link Bit Rate	(500k x2)
Packet Transmission Times	1ms per link
Rate/ Run length	10 e 8
Seed	400440917, 400473040
runs	2 * 12

Part 3 models an M/D/2 system with two parallel 500 kbps servers and one shared FCFS queue.

Packets are 500 bits long. Each packet is transmitted entirely by one server, so the service time is 1 ms.

The combined capacity of the two links is 1 Mbps, which corresponds to 2000 packets/s.

Arrival rates tested are 300, 500, 700, 900, 1100, 1300, 1500, 1600, 1700, 1800, 1850, and 1900 packets/s. The seeds are 400440917 and 400473040. Every run began with an empty system and ended after 10,000,000 completed packets. No warm-up period was removed. Delay was measured from arrival to the completion of transmission. The two seeds were averaged at each arrival rate.

B. Results:

	lambda	seed	processed	violations	probability	mean_delay_ms
	300	400440917	10000000	0	0.0000000000	1.013684
	300	400473040	10000000	0	0.0000000000	1.013731
	500	400440917	10000000	0	0.0000000000	1.037845
	500	400473040	10000000	0	0.0000000000	1.037884
	700	400440917	10000000	0	0.0000000000	1.076601
	700	400473040	10000000	0	0.0000000000	1.076647
	900	400440917	10000000	0	0.0000000000	1.135880
	900	400473040	10000000	0	0.0000000000	1.135868
	1100	400440917	10000000	0	0.0000000000	1.227573
	1100	400473040	10000000	0	0.0000000000	1.227586
	1300	400440917	10000000	0	0.0000000000	1.377858
	1300	400473040	10000000	0	0.0000000000	1.377868

1500	400440917	10000000	0	0.0000000000	1.655313
1500	400473040	10000000	0	0.0000000000	1.654188
1600	400440917	10000000	0	0.0000000000	1.900512
1600	400473040	10000000	0	0.0000000000	1.897610
1700	400440917	10000000	23	0.0000023000	2.312464
1700	400473040	10000000	0	0.0000000000	2.305079
1800	400440917	10000000	2457	0.0002457000	3.139502
1800	400473040	10000000	1977	0.0001977000	3.120127
1850	400440917	10000000	23442	0.0023442000	3.968301
1850	400473040	10000000	22432	0.0022432000	3.937036
1900	400440917	10000000	189899	0.0189899000	5.632419
1900	400473040	10000000	192705	0.0192705000	5.594379

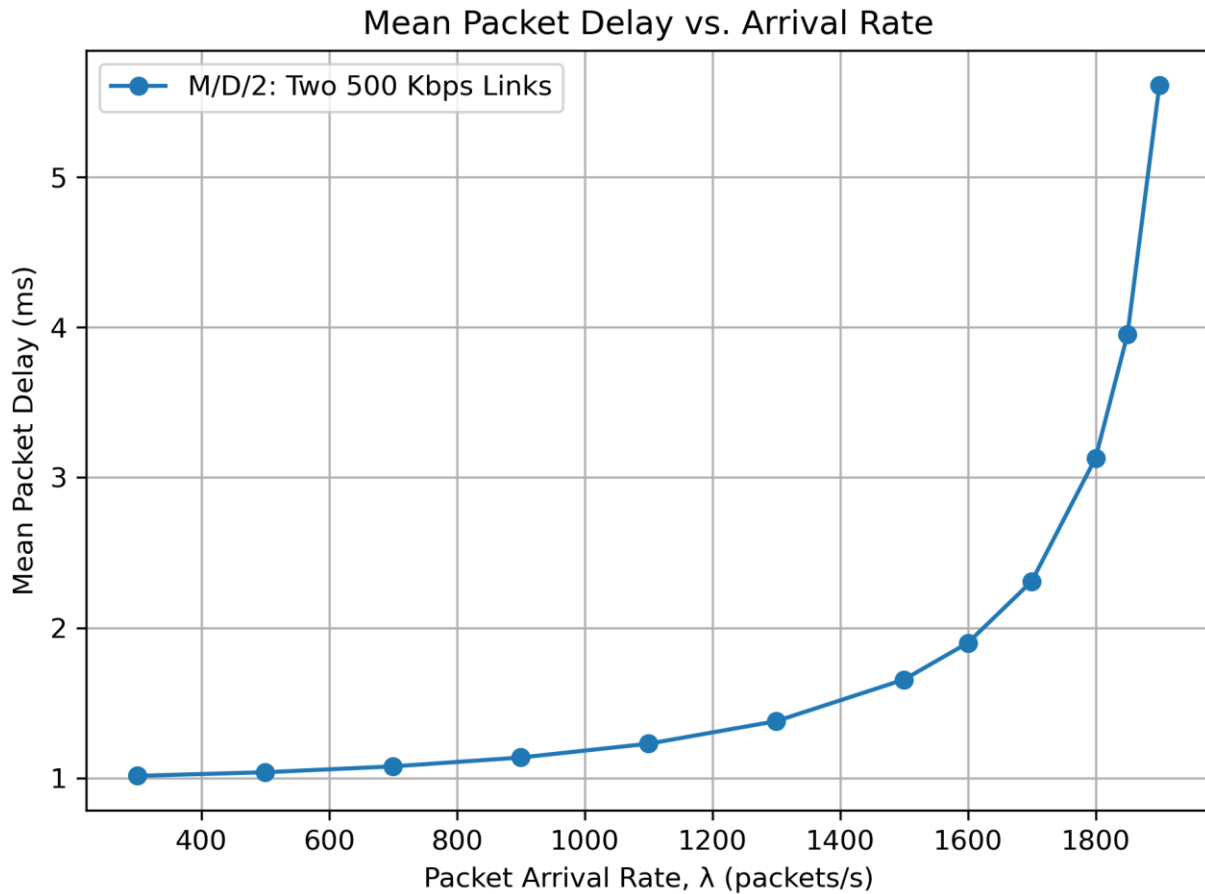


Figure 3. Mean packet delay for the M/D/2 system using two 500 kbps of links.

The measured delay increased from about 1.014 ms at 300 packets/s to about 5.613 ms at 1900 packets/s. The M/D/2 system exceeded the M/D/1 delay over the measured range because every packet required a longer individual transmission time, even though two packets could be served in parallel.

C. Discussion

At low loads, queuing is minimal, and the average latency—approximately 1 ms—primarily reflects transmission time. At $\lambda = 300$, the average latencies for the two random seeds were 1.013684 ms and 1.013731 ms, respectively; at $\lambda = 500$, they were 1.037845 ms and 1.037884 ms; at $\lambda = 700$, approximately 1.0766 ms; at $\lambda = 900$, approximately 1.1359 ms; at $\lambda = 1100$, approximately 1.2276 ms; at $\lambda = 1300$, approximately 1.3779 ms; and at $\lambda = 1500$, approximately 1.654–1.655 ms. In these cases, the number of violations was zero, indicating that no latencies exceeding 20 ms occurred. While the

two links can transmit two packets concurrently, they cannot aggregate bandwidth to complete a single packet faster.

As the arrival rate continues to increase, the average latency rises gradually, with the rate of increase accelerating as the load approaches the total service capacity of 2,000 packets per second. At $\lambda = 1600$, the average latency is approximately 1.898–1.901 ms; at $\lambda = 1700$, it is approximately 2.305–2.312 ms, with the first occurrences of 0–23 violations (a probability of approximately 0–0.00023%); at $\lambda = 1800$, the average latency is approximately 3.120–3.140 ms, and the violation probability is approximately 0.01977%–0.02457%; at $\lambda = 1850$, the average latency is approximately 3.937–3.968 ms, and the violation probability is approximately 0.2243%–0.2344%; at $\lambda = 1900$, the average latency is approximately 5.594–5.632 ms, and the violation probability is approximately 1.899%–1.927%. Given a total service capacity of 2000 packets/second, $\lambda = 1900$ corresponds to $\rho = 0.95$; while this remains below the stability limit, both the latency and the probability of exceeding 20 ms have risen significantly.

The results for both random seeds are very close across all data points, indicating that 10 million completed operations per run are sufficient to smooth out random fluctuations. Overall, this dataset exhibits trends similar to the M/D/2 model: at low loads, latency is primarily composed of a transmission time of approximately 1 ms; as the arrival rate approaches link capacity, queuing latency becomes the dominant component of total latency, causing a rapid increase in both average latency and the probability of exceeding 20 ms.

6. Part 4: Three-Switch Network

A. Experiment Settings :

Settings	Values
Model	Three-switch network, FCFS
Packet Length	1000 bits
Link 1 Bit Rate	2 Mbps
Link 2 Bit Rate	1 Mbps
Link 3 Bit Rate	1 Mbps
Link 1 Service Time	0.5 ms
Link 2/3 Service Time	1 ms
Arrival Rates	$\lambda_1 = 750$ pkt/s, $\lambda_2 = \lambda_3 = 500$ pkt/s
Routing Probability	p12 scanned 0 to 1
Stability Region	$1/3 < p_{12} < 2/3$
RUNLENGTH	10,000,000 packets
Seeds	400440917, 400473040
Runs	2×11

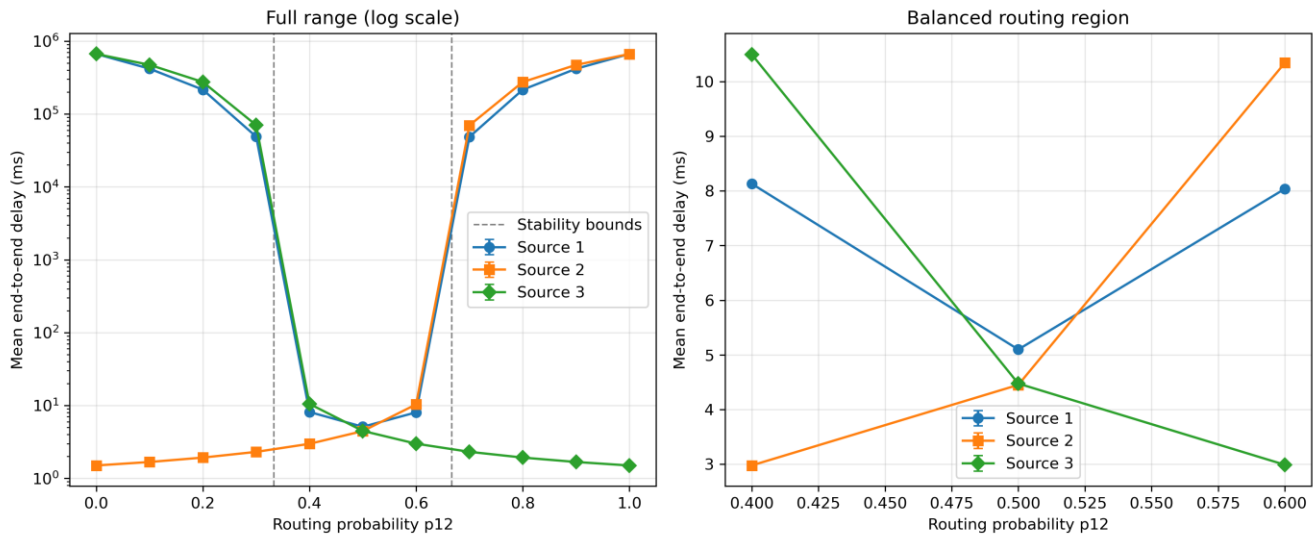
Part 4 uses a three-switch network. Each switch has its own FIFO queue and one server. All packets are 1000 bits. Switch 1 runs at 2 Mbps, so its service time is 0.5 ms. Switches 2 and 3 each run at 1 Mbps, so their service times are 1 ms. Local Poisson arrivals have rates of 750 packets/s at Switch 1 and 500 packets/s at both Switch 2 and Switch 3.

After a packet finish at Switch 1, it is routed to Switch 2 with probability p_{12} or to Switch 3 with probability $1 - p_{12}$. Each packet is forwarded only once, with no duplication. Propagation delay, processing delay, wireless errors, and retransmissions are ignored. A forwarded packet keeps its original arrival time and source ID. End-to-end delay is recorded only after the packet finishes at Switch 2 or Switch 3, and results are grouped by original source.

The p_{12} values scanned are 0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, and 1.0. Seeds 400440917 and 400473040 were used. Each run started empty and ended after 10,000,000 final departures. The stable range for p_{12} is between $1/3$ and $2/3$; outside this range one downstream switch becomes overloaded.

All packets were 1000 bits. Link 1 operated at 2 Mbps, giving a 0.5 ms service time, while Links 2 and 3 operated at 1 Mbps, giving 1 ms service times. Source 1 crossed Link 1 and was then routed to Link 2 with probability p_{12} or Link 3 with probability $1 - p_{12}$. Sources 2 and 3 entered the two-exit links directly.

B. results



	A	B	C	D	E	F	G	H	I	J	K
1	p12	seed	source1_c	source2_c	source3_c	source1_d	source2_d	source3_d	delay_ms		
2	0	400440917	3999097	3333093	2667810	665915	1.501932	665726			
3	0	400473040	4000766	3332470	2666764	666758.8	1.499806	666732.6			
4	0.1	400440917	4122705	3174285	2703010	418091.7	1.676782	472449.1			
5	0.1	400473040	4123996	3173726	2702278	418016.8	1.67527	472626.2			
6	0.2	400440917	4214167	3029397	2756436	215492.9	1.926375	274705.1			
7	0.2	400473040	4215307	3029098	2755595	215517.2	1.925324	274799.7			
8	0.3	400440917	4272887	2897685	2829428	48740.72	2.308748	70136.14			
9	0.3	400473040	4273825	2897674	2828501	48910.94	2.306656	70403.9			
10	0.4	400440917	4284793	2856487	2858720	8.138909	2.973904	10.5077			
11	0.4	400473040	4285765	2855951	2858284	8.124054	2.976872	10.49063			
12	0.5	400440917	4284797	2856481	2858722	5.09493	4.452427	4.464542			
13	0.5	400473040	4285765	2855951	2858284	5.100488	4.447091	4.483389			
14	0.6	400440917	4284797	2856481	2858722	8.018076	10.31188	2.981167			
15	0.6	400473040	4285765	2855948	2858287	8.058569	10.37775	2.992872			
16	0.7	400440917	4272687	2827637	2899676	48631.84	69931.54	2.307675			
17	0.7	400473040	4273958	2827012	2899030	48313.49	69558.73	2.313894			
18	0.8	400440917	4214251	2754612	3031137	215411.8	274443.1	1.924848			
19	0.8	400473040	4215367	2753460	3031173	215286.9	274567.9	1.92831			
20	0.9	400440917	4123523	2701837	3174640	417300.1	471538	1.674887			
21	0.9	400473040	4123954	2700420	3175626	417284.7	471806.1	1.677316			
22	1	400440917	3999795	2666715	3333490	665961.8	665418	1.499624			
23	1	400473040	4000421	2665378	3334201	665835.4	665952.6	1.500344			
24											

Figure 4. Correct Part 4 for end-to-end delays on a logarithmic scale. Dashed lines mark the theoretical stability limits $p_{12} = 1/3$ and $2/3$; the dotted line marks balanced routing.

At $p_{12} = 0.5$, the two-seed mean delays were 5.098 ms for Source 1, 4.450 ms for Source 2, and 4.474 ms for Source 3. The downstream offered loads are $500 + 750p_{12}$ packets/s on Link 2 and $500 + 750(1 - p_{12})$ packets/s on Link 3. Since each downstream link can serve 1000 packets/s, stability requires $1/3 <$

$p_{12} < 2/3$. Outside this interval, one downstream queue is unstable, so the finite-run mean delay becomes extremely large. The logarithmic axis exposes both the stable region and the overload behavior.

C. Discussion:

The routing probability p_{12} controls how much of Switch 1's forwarded traffic goes to Switch 2 and how much goes to Switch 3. When p_{12} increases, Switch 2 receives more forwarded packets and Switch 3 receives fewer. As a result, packets originating at Switch 2 experience increasing delay, while packets originating at Switch 3 experience decreasing delay.

For the system to remain stable, both downstream switches must have utilization below 1. With local arrival rates of 750 packets/s at Switch 1 and 500 packets/s at both Switch 2 and Switch 3, and with service rates of 2000 packets/s at Switch 1 and 1000 packets/s at Switches 2 and 3, the stable range is approximately p_{12} greater than $1/3$ and p_{12} less than $2/3$. Outside this range, one downstream switch becomes overloaded.

At $p_{12} = 0.0$, Switch 3 has utilization 1.25 while Switch 2 has utilization 0.50. This produces extremely large delays for Source 3 and Source 1: about 666,229 ms and 666,337 ms, respectively. Source 2, by contrast, sees only about 1.501 ms. At $p_{12} = 1.0$, the situation reverses. Switch 2 has utilization 1.25 and Switch 3 has utilization 0.50, giving Source 2 about 665,685 ms, Source 1 about 665,899 ms, and Source 3 about 1.500 ms.

Inside the stable region, the delays are much smaller, and the trade-off is clear. At $p_{12} = 0.4$, Switch 2 has utilization 0.80 and Switch 3 has utilization 0.95. Source 2 has a mean delay of about 2.975 ms, while Source 3 has about 10.499 ms. At $p_{12} = 0.6$, the loads swap: Switch 2 has utilization 0.95 and Switch 3 has utilization 0.80. Source 2 rises to about 10.345 ms, while Source 3 drops to about 2.987 ms.

The most balanced point is $p_{12} = 0.5$. At this point both downstream switches carry 875 packets/s and both have utilization 0.875. The mean delays are about 5.098 ms for Source 1, 4.450 ms for Source 2, and 4.474 ms for Source 3. This is the minimum for Source 1 and the most even distribution overall. Source 1 traffic must pass through Switch 1 and then one downstream switch, so its delay is lowest when the downstream load is split evenly. Unequal splitting concentrates forwarded traffic on one branch, pushing that branch toward saturation and raising Source 1's delay. For example, at $p_{12} = 0.3$ and $p_{12} = 0.7$, Source 1 delays jump to about 48,826 ms and 48,473 ms, because one downstream switch is near or above capacity.

Simulation accuracy

The results use two random seeds and 10,000,000 final departures per seed. In the balanced region, seed-to-seed variation is small. At $p_{12} = 0.5$, the min–max ranges are about 5.095–5.100 ms for Source 1, 4.447–4.452 ms for Source 2, and 4.465–4.483 ms for Source 3. Near the stability boundaries, variation increases. At $p_{12} = 0.4$, Source 3 ranges from about 10.491 to 10.508 ms. At $p_{12} = 0.6$, Source 2 ranges from about 10.312 to 10.378 ms. At $p_{12} = 0.3$, Source 3 ranges from about 70,136 to 70,404 ms. At $p_{12} = 0.7$, Source 2 ranges from about 69,559 to 69,932 ms. These spreads support the congestion trend but show that near-critical queues need longer runs for precise estimates. Since no 50M validation was performed, the accuracy statement is limited to 10M final departures per seed.

7. Part 5: FCFS Voice and Data

A. Experiment Settings :

Settings	Values
Model	Integrated voice/data, FCFS
Data traffic	Poisson arrivals, exponentially distributed

	service time with mean 40 ms
Link Bit Rate	1Mbps
Packet Transmission Times	1.776 ms
Rate/Run length	1 e 6
Seed	400440917, 400473040
Voice traffic	G.711 64 kbps, periodic arrivals, fixed inter-packet interval
Data arrival rate λ_D	1:22
runs	2 * 13

Part 5 simulates voice and data traffic sharing one 1 Mbps link and one FCFS queue. Both classes wait in the same buffer, and packets are served in arrival order. The buffer is treated as unlimited, and no packets are dropped.

Voice packets are generated every 20 ms. The voice codec rate is 64 kbps, so each packet carries 1280 bits of payload. Adding a 62-byte header gives 496 bits of overhead, so the total voice packet size is $1280 + 496 = 1776$ bits. On a 1 Mbps link, this takes $1776 / 1,000,000 = 0.001776$ s, or 1.776 ms, to transmit. Since one voice packet arrives every 20 ms, the voice arrival rate is $1 / 0.020 = 50$ packets/s. The voice load is therefore $50 \times 0.001776 = 0.0888$, which is 8.88% of the link capacity.

Data arrivals are Poisson, and data service times are exponential with a mean of 40 ms, which is 0.040 s. If the data arrival rate is λ_d packets/s, the data load is $0.040 \times \lambda_d$. The total offered load is therefore $0.0888 + 0.040 \times \lambda_d$. For the system to remain stable, the total load must be below 1. This gives $0.0888 + 0.040 \times \lambda_d < 1$, which means $\lambda_d < 22.78$ packets/s.

The data arrival rates tested are 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 20, 21, and 22 packets/s. Seeds 400440917 and 400473040 were used. Each run started with an empty system and ended after 1,000,000 total voice and data departures. Voice traffic was initialized at $t=0$, whereas the first data packet was generated according to the exponential arrival process. Delay was measured from arrival to transmission completion, and voice and data delays were computed separately. No warm-up period was removed.

B. results:

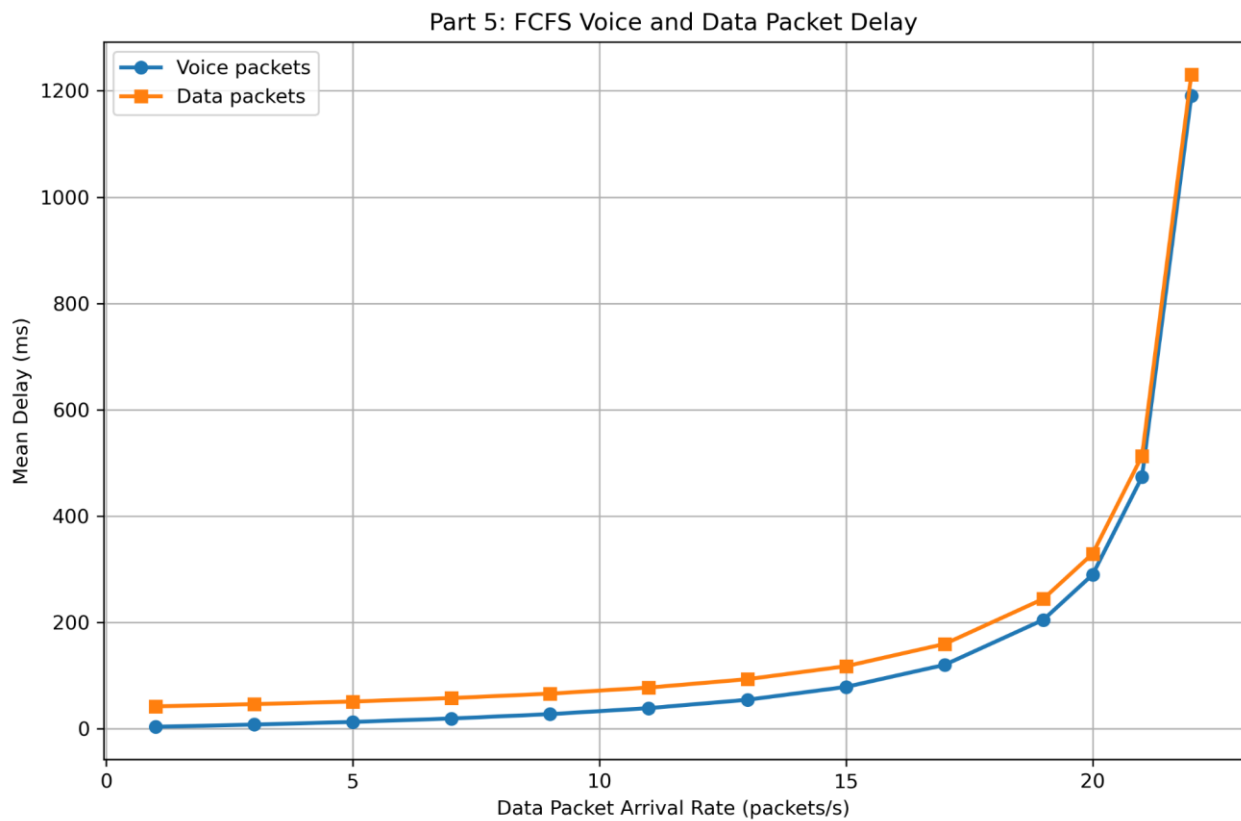


Figure 5. Exact seed-averaged voice and data delays under FCFS.

data_lambd	seed	voice_complet	data_complete	voice_delay_m	data_delay_m
------------	------	---------------	---------------	---------------	--------------

	a		ed	d	s	s
	1.0	400440917	980691	19309	3.56586565	41.99152113
	1.0	400473040	980526	19474	3.56255766	41.97638153
	3.0	400440917	943761	56239	7.71372969	46.14561499
	3.0	400473040	943527	56473	7.73674187	46.23787910
	5.0	400440917	909255	90745	12.67983293	50.87475800
	5.0	400473040	909163	90837	12.83157171	51.36409321
	7.0	400440917	877297	122703	19.08321983	57.42232325
	7.0	400473040	877251	122749	19.29656737	57.85659952
	9.0	400440917	847801	152199	27.30242965	65.81164228
	9.0	400473040	847480	152520	27.50291764	66.03008941
	11.0	400440917	819807	180193	38.47710377	77.09437173
	11.0	400473040	819806	180194	38.90021282	77.60392017
	13.0	400440917	793621	206379	54.24105479	92.98241357
	13.0	400473040	793916	206084	54.54395589	93.23177916

15.0	400440917	769306	230694	78.71929040	117.78580630
15.0	400473040	769530	230470	78.53017329	117.36652634
17.0	400440917	746380	253620	120.70757256	160.06999421
17.0	400473040	746574	253426	119.56866203	158.79773734
19.0	400440917	724632	275368	206.27532845	245.94291556
19.0	400473040	724848	275152	203.44004483	242.46067399
20.0	400440917	714377	285623	289.22187792	328.85593791
20.0	400473040	714327	285673	290.58197604	329.62725856
21.0	400440917	704369	295631	468.41189658	507.96449454
21.0	400473040	704213	295787	478.78356863	517.81782911
22.0	400440917	694578	305422	1019.05789912	1056.46023406
22.0	400473040	694384	305616	1361.53940006	1404.58703036

data_lambda**voice_delay_ms****data_delay_ms**

1.0

3.5642116550000003

41.98395133

3.0	7.72523578	46.191747045
5.0	12.755702320000001	51.119425605
7.0	19.1898936	57.639461385000004
9.0	27.402673645	65.92086584500001
11.0	38.688658294999996	77.34914595000001
13.0	54.39250534	93.107096365
15.0	78.624731845	117.57616632
17.0	120.138117295	159.43386577500002
19.0	204.85768664	244.201794775
20.0	289.90192698	329.24159823499997
21.0	473.59773260500003	512.891161825
22.0	1190.29864959	1230.52363221

At 22 data packets/s, voice and data delays were 1190.299 ms and 1230.524 ms. Total utilization was approximately $0.0888 + 0.04 \lambda_D$, which equals 0.9688 at 22 packets/s. This proximity to the 22.78 packets/s stability boundary explains the rapid delay growth and large seed-to-seed variability.

C. Discussion

At the lowest data arrival rate, 1.0 packet/s, each run completes about 1,000,000 total voice and data transmissions per seed. The mean voice delay is about 3.564 ms and the mean data delay is about 41.984 ms. As the data arrival rate increases, both delays grow slowly at first and then accelerate sharply at high rates. At 17 data packets/s, voice delay is about 120.138 ms and data delay is about 159.434 ms. At 19 packets/s, they rise to about 204.858 ms and 244.202 ms. At 20 packets/s, they reach about 289.902 ms and 329.242 ms. At 21 packets/s, they are about 473.598 ms and 512.891 ms. By 22 packets/s, voice delay reaches about 1190.299 ms and data delay reaches about 1230.524 ms. Moving from 21 to 22 packets/s, voice delay increases by roughly a factor of 2.51, and data delay increases by roughly a factor of 2.40. This steep rise occurs because the total offered load at 22 packets/s is about 0.9688, leaving very little spare capacity to clear queues formed by data bursts and long data transmissions.

Data delays are consistently higher than voice delays by about 38 to 40 ms. At 1.0 packet/s the difference is about 38.42 ms, and at 22 packets/s it is about 40.23 ms. The main reason is the difference in service time: data packets average about 40 ms per transmission, while voice packets take only about 1.776 ms. Under FCFS scheduling, packet type does not affect service order. A voice packet therefore cannot bypass data packets that have already entered the queue. This shows that FCFS does not provide adequate delay protection for real-time voice traffic as the data load grows.

Seed-to-seed agreement is good at low and moderate loads, but the two seeds diverge noticeably at the highest load. At 22 data packets/s, seed 400440917 gives a mean voice delay of about 1019.058 ms and a mean data delay of about 1056.460 ms, while seed 400473040 gives about 1361.539 ms for voice and about 1404.587 ms for data. The summary values of 1190.299 ms for voice and 1230.524 ms for data are the averages of the two seeds. This large spread confirms that near capacity, random bursts can still cause significant run-to-run variability.

8. Part 6: Non-Preemptive Voice Priority

A. Experiment Settings :

Settings	Values
Model	Integrated voice/data, non-preemptive voice priority
Data traffic	Poisson arrivals, exponentially distributed service time with mean 40 ms
Link Bit Rate	1Mbps
Packet Transmission Times	1.776ms
Rate/Run length	1 e 6
Seed	400440917, 400473040
Voice traffic	G.711 64 kbps, periodic arrivals, fixed inter-packet interval

Part 6 uses the same traffic parameters as Part 5, but voice packets receive non-preemptive priority over data packets. The link rate is 1 Mbps, and the packet sizes and service times are unchanged.

Each voice packet is generated every 20 ms. The 64 kbps codec produces 1280 bits of payload per packet, and the 62-byte header adds 496 bits. The total voice packet size is therefore $1280 + 496 = 1776$ bits. On the 1 Mbps link, this gives a fixed voice transmission time of $1776 / 1,000,000 = 0.001776$ s, or 1.776 ms. The voice arrival rate is $1 / 0.020 = 50$ packets/s, so the voice load is $50 \times 0.001776 = 0.0888$, which is 8.88%.

Data arrivals are Poisson, and data service times are exponential with a mean of 40 ms, or 0.040 s. At a data arrival rate of λ_d packets/s, the data load is $0.040 \times \lambda_d$. The total offered load is $0.0888 + 0.040 \times \lambda_d$. Stability requires this total to stay below 1, so $0.0888 + 0.040 \times \lambda_d < 1$, which gives $\lambda_d < 22.78$ packets/s.

Two separate queues are used, one for voice and one for data. Whenever the link becomes free, the scheduler checks the voice queue first. If a voice packet is waiting, the oldest voice packet is served. If the voice queue is empty, the oldest data packet is served. A data packet already in transmission is never interrupted, so the priority is non-preemptive.

The data arrival rates tested are the same as in Part 5: 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 20, 21, and 22 packets/s. Seeds 400440917 and 400473040 were used. Each run started empty and ended after 1,000,000 total departures. Delay was measured from arrival to transmission completion and grouped by traffic type. The FCFS baseline comes from the completed Part 5 results. No warm-up period was removed.

B. results:

data_lambda	seed	voice_completed	data_completed	voice_delay_ms	data_delay_ms
1.0	400440917	980691	19309	3.48142912	42.17566076
1.0	400473040	980526	19474	3.47795586	42.15823138
3.0	400440917	943761	56239	6.91628790	46.72914116
3.0	400473040	943527	56473	6.93912896	46.82313861

5.0	400440917	909255	90745	10.28122773	51.93509332
5.0	400473040	909163	90837	10.41146489	52.44355131
7.0	400440917	877297	122703	13.77447302	59.10791881
7.0	400473040	877251	122749	13.95728529	59.55564131
9.0	400440917	847801	152199	17.23389004	68.29989064
9.0	400473040	847480	152520	17.43659809	68.52282395
11.0	400440917	819807	180193	20.74622755	80.67620614
11.0	400473040	819806	180194	21.03111677	81.21065078
13.0	400440917	793621	206379	24.23025040	98.10889795
13.0	400473040	793916	206084	24.45667048	98.36049149
15.0	400440917	769306	230694	27.77820603	125.31839773
15.0	400473040	769532	230468	27.96000679	124.84032815
17.0	400440917	746382	253618	31.28187819	171.71393830
17.0	400473040	746574	253426	31.43581715	170.30760155

19.0	400440917	724632	275368	34.84348307	265.94861702
19.0	400473040	724850	275150	35.00850034	262.11647486
20.0	400440917	714380	285620	36.60559560	356.93930926
20.0	400473040	714339	285661	36.75095564	357.76315513
21.0	400440917	704371	295629	38.36460272	553.49972698
21.0	400473040	704223	295777	38.56615839	564.29239607
22.0	400440917	694582	305418	40.12930108	1155.45522000
22.0	400473040	694404	305596	40.37388641	1537.49719958

P5 vs p6	data_lamb da	voice_p5	data_p5	voice_p6	data_p6
	1.0	3.564211655000 0003	41.98395133	3.47969248999999 97	42.16694606999 9995
	3.0	7.72523578	46.191747045	6.92770843	46.776139885

5.0	12.75570232000 0001	51.119425605	10.34634631	52.189322315
7.0	19.1898936	57.639461385000 004	13.865879155	59.33178006
9.0	27.402673645	65.920865845000 01	17.3352440649999 98	68.411357295
11.0	38.68865829499 9996	77.349145950000 01	20.88867216	80.94342846
13.0	54.39250534	93.107096365	24.34346044	98.23469472
15.0	78.624731845	117.57616632	27.86910641	125.0793629400 0001
17.0	120.138117295	159.43386577500 002	31.35884767	171.0107699249 9998
19.0	204.85768664	244.201794775	34.925991705	264.03254594
20.0	289.90192698	329.24159823499 997	36.67827562	357.3512321949 9997
21.0	473.5977326050 0003	512.891161825	38.4653805549999 96	558.896061525
22.0	1190.29864959	1230.52363221	40.2515937449999	1346.47620979

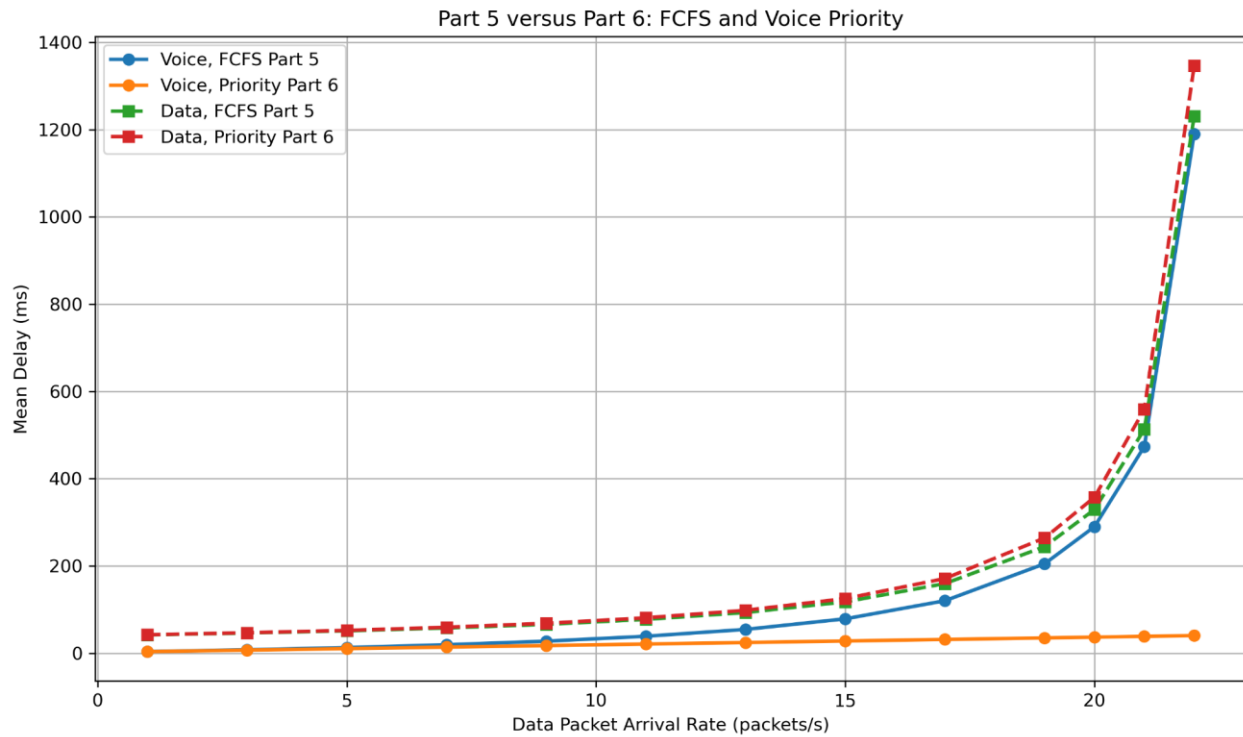


Figure 6. Exact seed-averaged comparison of FCFS and non-preemptive voice priority.

At 22 data packets/s, voice priority reduced mean voice delay from 1190.299 ms to 40.252 ms, a reduction of approximately 96.6%. Data delay increased from 1230.524 ms to 1346.476 ms. Priority protected the delay-sensitive traffic class by transferring more waiting time to data traffic. Voice delay did not reach the 1.776 ms service time because voice could still wait for the residual service time of a data packet already transmitting.

C. Discussion

Voice priority becomes more beneficial as the data arrival rate rises. At 13 data packets/s, mean voice delay drops from about 54.393 ms under FCFS to about 24.343 ms with priority, while mean data delay rises from about 93.107 ms to about 98.235 ms. At 19 packets/s, voice delay falls from about 204.858 ms to about 34.926 ms, and data delay rises from about 244.202 ms to about 264.033 ms. At 21

packets/s, voice delay drops from about 473.598 ms to about 38.465 ms, and data delay rises from about 512.891 ms to about 558.896 ms. By 22 packets/s, mean voice delay decreases from about 1190.299 ms under FCFS to about 40.252 ms with priority, a reduction of roughly 96.6%. At the same point, mean data delay increases from about 1230.524 ms to about 1346.476 ms, a rise of roughly 9.4%.

The reason is that waiting voice packets are selected before waiting for data packets whenever the link becomes free. However, because the priority is non-preemptive, a voice packet still has to wait for any data packet already in service. This is why voice delay does not fall all the way to the 1.776 ms service time. The improvement in voice delay comes at the cost of higher data delay: data packets now wait longer because voice packets are served first.

At high data rates, both data delay curves rise sharply because the total offered load approaches 1. With $\lambda_d = 22$ packets/s, the calculated utilization reaches 0.9688, leaving only about 3.12% of the link capacity unused on average. Priority scheduling only shifts waiting time from voice to data; it does not add link capacity. In summary, voice priority is advantageous when low voice delay matters more than low data delay. Priority scheduling substantially lowers the mean delay experienced by voice traffic, but it neither establishes a worst-case delay bound nor changes the overall stability condition.

9. Conclusions

The experiments show that average delay is shaped by several factors beyond total link capacity: utilization, per-server service rate, routing decisions, and scheduling policy. The delay-tail requirement proved more restrictive than the stability limit. Balancing traffic across downstream switches reduced overall delay, while non-preemptive voice priority significantly lowered voice delay at the cost of higher data delay. These results highlight that capacity alone does not determine performance; how traffic is distributed and scheduled can be just as important.

